LAPORAN PRAKTIKUM

IOT 2

MODUL 6



Disusun Oleh:

Nama : Reza Aurelia Saputri

NPM : 25782080

Prodi : TRI C

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

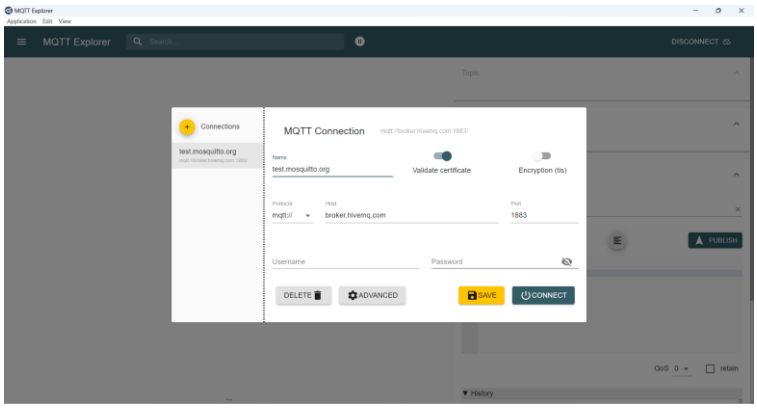
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

PRAKTIKUM 1

Halaman ini digunakan untuk mengatur parameter penghubung antara aplikasi MQTT Explorer di komputer Anda dengan server broker MQTT publik yang sedang digunakan pada praktikum ini.

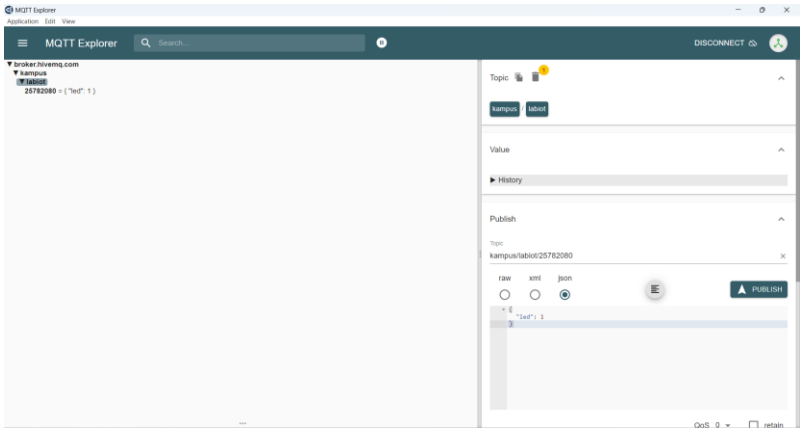


Panel Kiri (Topic Tree): Menampilkan hierarki direktori topik yang masuk dari *broker*, yaitu folder kampus → labiot → 25782080 dengan nilai data terakhir {"led": 1}.

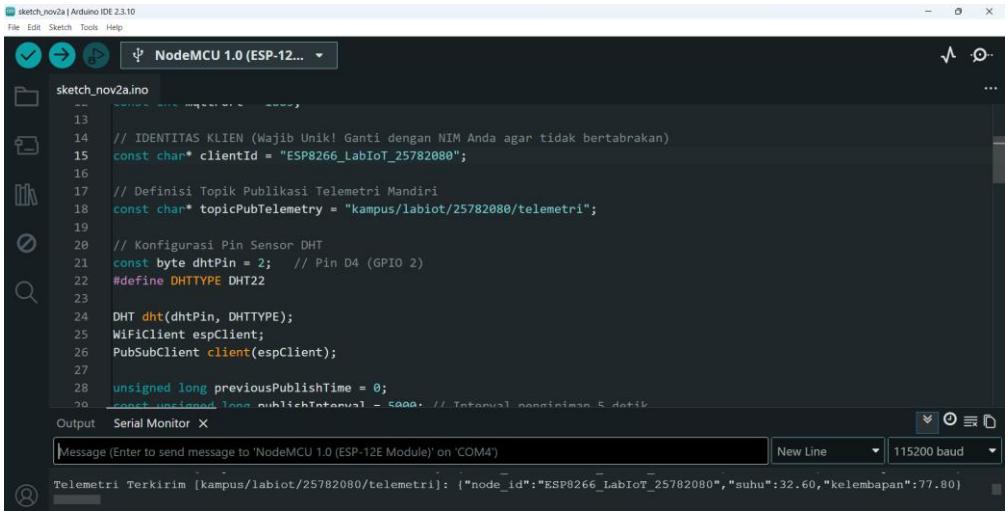
Panel Kanan Atas (Topic Path): Menampilkan jalur direktori aktif yang sedang dipilih, yaitu kampus / labiot.

Panel Kanan Bawah (Area Publish):

- **Kotak Topic:** Berisi alamat tujuan pengiriman pesan, yaitu kampus/labiot/25782080.
- **Format Payload:** Menggunakan format **json**.
- **Isi Pesan (Payload):** Berisi data string JSON {"led": 1}.
- **Tombol PUBLISH:** Digunakan untuk mengirimkan data atau perintah tersebut secara langsung ke *broker* MQTT.



- **Bagian Kode Program (Atas):**
    - const char* clientId = "ESP8266_LabIoT_25782080";: Mendefinisikan identitas unik klien MQTT menggunakan NIM Anda (**25782080**) agar tidak bertabrakan dengan perangkat lain di broker publik.
    - const char* topicPubTelemetry = "kampus/labiot/25782080/telemetri";: Menentukan jalur topik MQTT khusus tempat data sensor akan dipublikasikan.
    - Konfigurasi pin sensor DHT diatur pada pin **D4 (GPIO 2)** dengan jenis sensor **DHT22**.
    - Inisialisasi objek komunikasi jaringan menggunakan WiFiClient dan pustaka MQTT PubSubClient.
- **Bagian Serial Monitor (Bawah):**
    - Menampilkan log status pengiriman data secara berkala: Telemetri Terkirim [kampus/labiot/25782080/telemetri]: {"node_id":"ESP8266_LabIoT_25782080","suhu":32.60,"kelembapan":77.80}.
    - Ini membuktikan bahwa program berhasil membaca suhu lingkungan sebesar **32.60°C** dan kelembapan **77.80%** dari sensor DHT22, lalu sukses mengirimkannya dalam format paket data JSON ke *broker* melalui topik telemetri NIM Anda.



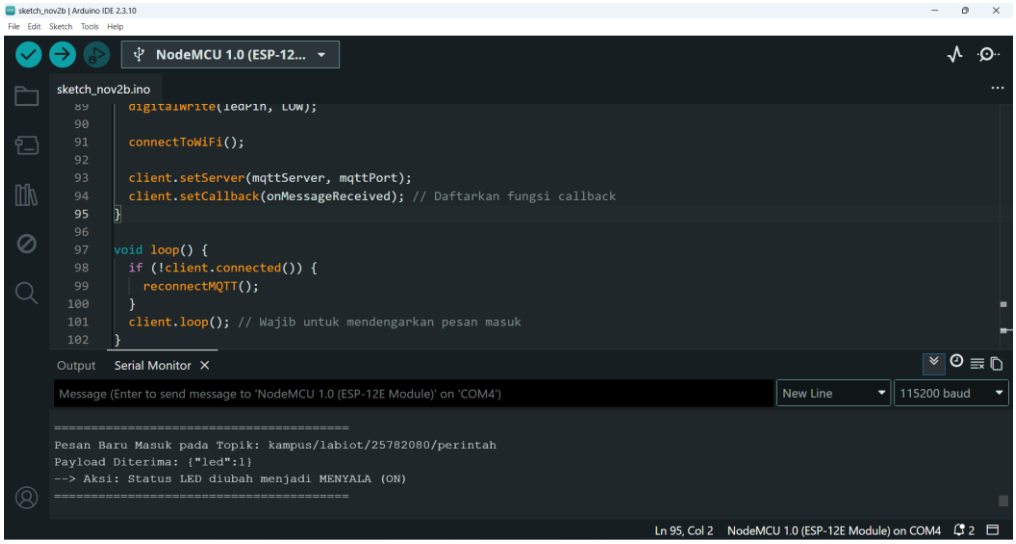
PRAKTIKUM 2

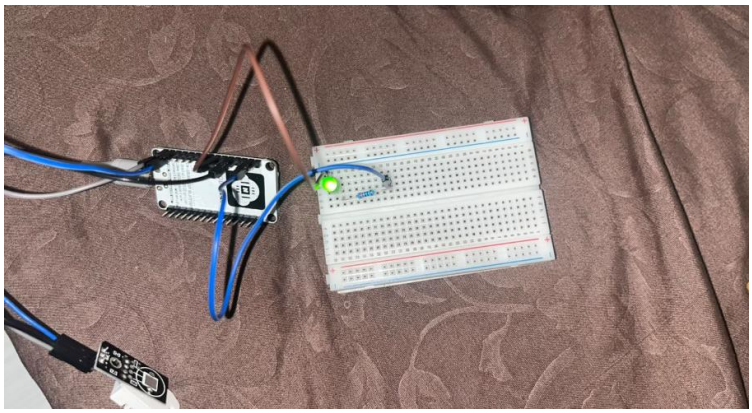
Bagian Kode Program (Atas):

- Menunjukkan fungsi utama loop() yang memuat fungsi client.loop(); // Wajib untuk mendengarkan pesan masuk. Baris kode ini sangat penting agar mikrokontroler NodeMCU terus siaga mendengarkan perintah kendali dari luar secara *real-time*.
- Terdapat juga fungsi reconnectMQTT(); di dalam blok kondisi if (!client.connected()), yang berfungsi untuk memulihkan koneksi secara otomatis jika hubungan ke *broker* terputus.

Bagian Serial Monitor (Bawah):

- Menunjukkan log aktivitas ketika mikrokontroler berhasil menerima pesan kendali dari *broker* MQTT.
- Topik Tujuan: kampus/labiot/25782080/perintah.
- Payload yang Diterima: {"led":1} (perintah untuk menyalakan LED).
- Eksekusi Fisik: --> Aksi: Status LED diubah menjadi MENYALA (ON). Log ini membuktikan bahwa sistem komunikasi dua arah (kendali jarak jauh) pada praktikum Anda telah berjalan dengan sukses dan merespons perintah dengan benar.

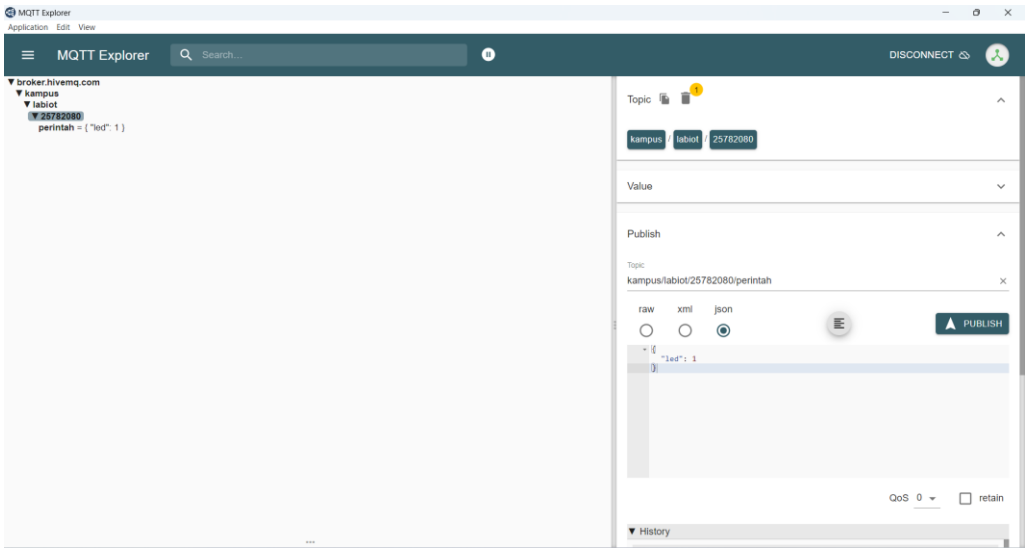




menampilkan antarmuka aplikasi **MQTT Explorer** saat terhubung ke *broker* publik (broker.hivemq.com), yang menunjukkan proses pengujian pengiriman perintah pengendalian perangkat secara langsung.

Berikut adalah penjelasan detail mengenai bagian-bagian yang terlihat pada gambar:

1. **Panel Kiri (Topic Tree / Pohon Topik):**
    - Menampilkan struktur direktori pesan MQTT yang sedang aktif berdasarkan hirarki topik.
    - Terdapat cabang folder: kampus → labiot → 25782080 (Nomor NIM Anda).
    - Di bawah direktori tersebut, terdapat sub-topik **perintah** yang menyimpan payload data terakhir berupa {"led": 1}.
2. **Panel Kanan Bawah (Area Publish):**
    - **Kotak Topic:** Menunjukkan alamat jalur lengkap tempat pesan akan dikirim, yaitu **kampus/labiot/25782080/perintah**.
    - **Format Payload:** Dipilih format berbasis **json**.
    - **Isi Pesan (Payload):** Berisi data string JSON {"led": 1} yang berfungsi sebagai instruksi untuk menyalakan LED pada perangkat mikrokontroler NodeMCU.
    - **Tombol PUBLISH:** Tombol untuk mengirimkan paket data perintah tersebut secara *real-time* melalui *broker* menuju perangkat keras yang mendengarkan (*subscribe*) pada topik yang sama.



PRAKTIKUM 3

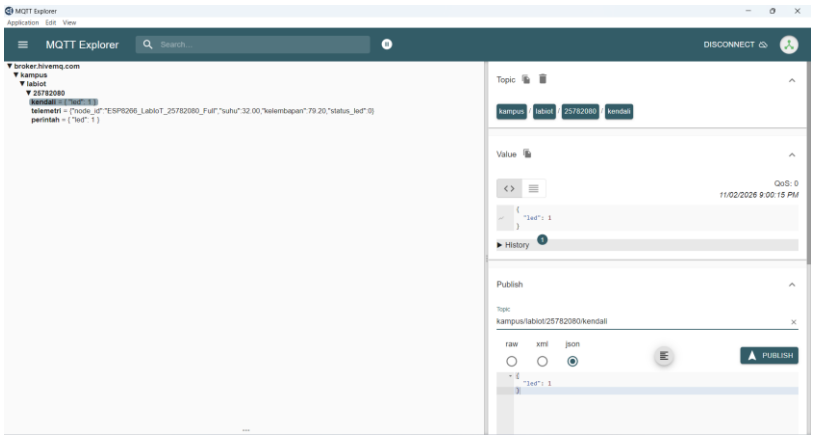
menampilkan antarmuka aplikasi **MQTT Explorer** yang terhubung ke broker publik (broker.hivemq.com), yang menunjukkan struktur hierarki topik dan proses pengujian kendali jarak jauh pada perangkat IoT Anda.

Berikut adalah penjelasan detail mengenai bagian-bagian yang terlihat pada gambar:

- **Panel Kiri (Topic Tree / Pohon Topik):**
    - Menunjukkan struktur direktori pesan MQTT yang aktif berdasarkan NIM Anda (25782080) di bawah folder kampus/labiot.
    - Terdapat tiga sub-topik utama yang terekam:
        1. **kendali**: Menyimpan nilai payload terakhir {"led": 1} yang dikirimkan ke perangkat.
        2. **telemetri**: Menampilkan paket data sensor berkala yang dikirim oleh mikrokontroler, berisi {"node_id":"ESP8266_LabIoT_25782080_Full","suhu":32.00,"kelembapan": 79.20,"status_led":0}.

3. **perintah**: Jalur alternatif atau riwayat topik perintah sebelumnya yang berisi {"led": 1}.

- **Panel Kanan Atas (Detail Topik & Nilai):**
    - Menampilkan jalur topik yang sedang dipilih, yaitu kampus / labiot / 25782080 / kendali dengan nilai payload format JSON {"led": 1}.
- **Panel Kanan Bawah (Area Publish):**
    - **Kotak Topic:** Berisi alamat tujuan pengiriman pesan, yaitu kampus/labiot/25782080/kendali.
    - **Format Payload:** Menggunakan format **json** dengan isi data berupa {"led": 1}.
    - **Tombol PUBLISH:** Digunakan untuk mengirimkan instruksi pengendalian tersebut secara *real-time* melalui broker agar diterima oleh mikrokontroler NodeMCU.



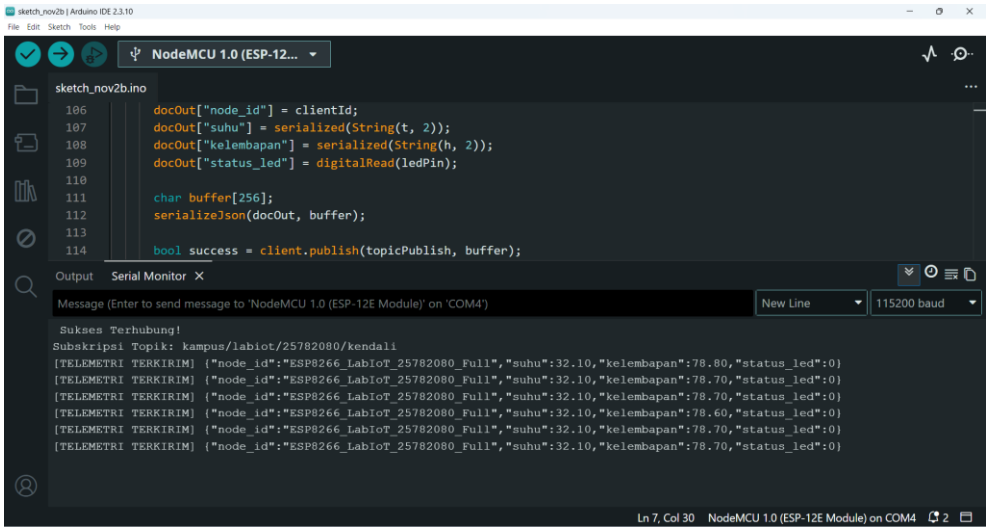
Bagian Kode Program (Atas):

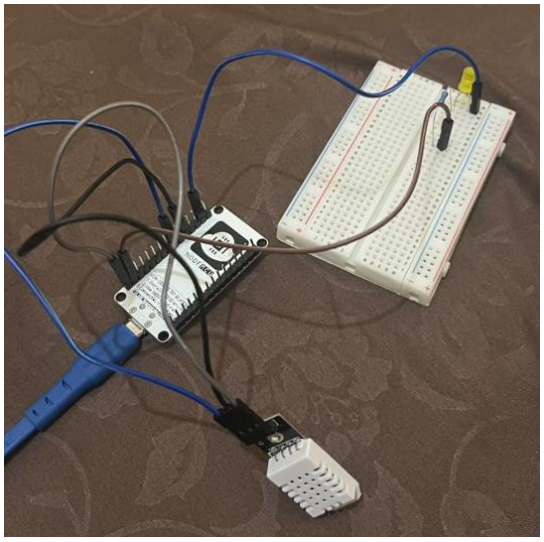
- Menunjukkan proses penyusunan data telemetri ke dalam objek JSON menggunakan pustaka ArduinoJson (docOut), yang mencakup parameter node_id, suhu, kelembapan, dan status_led.
- Berisi perintah serializeJson(docOut, buffer); untuk mengubah objek dokumen JSON tersebut menjadi bentuk string array (*buffer*) sebelum akhirnya dikirimkan melalui fungsi client.publish(topicPublish, buffer);.

Bagian Serial Monitor (Bawah):

- Menampilkan status koneksi awal: Sukses Terhubung! dan informasi subskripsi topik kendali pada kampus/labiot/25782080/kendali.

- Menampilkan log pengiriman data berkala: [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782080_Full","suhu":32.10,"kelembapan":78.80,"status_led":0}.
- Hal ini membuktikan bahwa mikrokontroler NodeMCU berhasil membaca sensor secara stabil, mengemas data dalam format JSON dengan benar, dan mempublikasikannya ke *broker* MQTT melalui jalur topik NIM





Latihan 1: Analisis Arsitektur & Perbandingan Protokol

1. **Efisiensi Penggunaan Bandwidth (Paket Seluler):**
    - **Penjelasan:** MQTT dirancang khusus sebagai protokol *lightweight* (ringan) berbasis *Publish-Subscribe* yang memiliki ukuran header sangat kecil (minimum hanya 2

byte). Sebaliknya, HTTP menggunakan format teks berbasis *request-response* dengan header teks yang sangat panjang (seperti *User-Agent*, *Host*, *Accept*, dll.) pada setiap pengiriman data. Hal ini membuat MQTT mengonsumsi kuota data seluler jauh lebih sedikit dibandingkan HTTP.

2. **Beban Komputasi Mikrokontroler (CPU Overhead):**
    - **Penjelasan:** MQTT menjaga koneksi TCP tetap terbuka secara berkelanjutan (*persistent connection*), sehingga mikrokontroler tidak perlu berulang kali melakukan proses *handshake* TCP yang berat setiap kali ingin mengirim atau menerima data. Pada HTTP, setiap permintaan memerlukan pembukaan dan penutupan koneksi baru (atau header *Keep-Alive* yang kompleks), yang membebani kapasitas RAM dan siklus CPU ESP8266 yang terbatas.
3. **Distribusi Data Simultan (1-to-N Distribution):**
    - **Penjelasan:** MQTT menggunakan sistem *Broker* berbasis *topic tree* yang mendukung pola komunikasi *one-to-many* secara native. Satu pesan yang dipublikasikan (*publish*) ke sebuah topik dapat langsung disebarkan secara otomatis oleh broker ke puluhan atau ratusan perangkat lain yang berlangganan (*subscribe*) pada topik tersebut tanpa membebani pengirim. Pada HTTP, pengirim harus melakukan pengiriman berulang (*multiple HTTP requests*) satu per satu ke setiap klien penerima.

Latihan 2: Analisis Hierarki Topik & Karakter Wildcard

Berdasarkan struktur topik dasar pabrik/<line>/<mesin>/<sensor>, berikut ekspresi *wildcard* yang tepat:

1. **Memantau parameter suhu dari seluruh mesin di semua lini produksi:**
    - **Topik Subskripsi:** pabrik/+/+/suhu
    - *Penjelasan:* Menggunakan karakter + (single-level wildcard) untuk mencakup semua lini dan semua mesin, diakhiri dengan spesifikasi sensor suhu.
2. **Memantau seluruh parameter sensor khusus untuk mesin m1 di lini line2:**
    - **Topik Subskripsi:** pabrik/line2/m1/#

- *Penjelasan:* Mengunci jalur pada line2 dan mesin m1, lalu menggunakan # (multi-level wildcard) untuk mencakup semua sub-parameter sensor di bawahnya.

3. **Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan mesin:**
   - **Topik Subskripsi:** pabrik/#
   - *Penjelasan:* Menggunakan multi-level wildcard # di tingkat akar (*root*) untuk mendengarkan seluruh data yang masuk ke dalam hierarki direktori pabrik.

Latihan 3: Implementasi Serialisasi Data Dinamis

Untuk menyertakan parameter uptime_detik dan ip_lokal ke dalam objek JSON telemetri, tambahkan baris kode berikut ke dalam fungsi pembentukan JSON (di dalam fungsi loop() sebelum fungsi serializeJson):

C++

```
// Tambahan Parameter Dinamis Praktikum
docOut["uptime_detik"] = millis() / 1000;
docOut["ip_lokal"]     = WiFi.localIP().toString();
```

Sehingga blok penyusunan JSON lengkapnya menjadi seperti ini:

C++

```
StaticJsonDocument<256> docOut;
docOut["node_id"]      = clientId;
docOut["suhu"]         = serialized(String(t, 2));
docOut["kelembapan"]   = serialized(String(h, 2));
docOut["status_led"]   = digitalRead(ledPin);
docOut["uptime_detik"] = millis() / 1000;
```

Latihan 4: Analisis Mekanisme Reconnect & Loop Blocking

1. **Konsekuensi Teknis Penyisipan delay(6000) di dalam loop():**
   - **Penjelasan:** Fungsi client.loop() berfungsi untuk memelihara koneksi jaringan (mengirim *ping* berkala) dan memproses paket data masuk (*callback*) yang

dikirimkan oleh broker. Jika pengembang menyisipkan delay(6000) (jeda 6 detik), mikrokontroler akan mengalami *blocking* total pada eksekusi program. Akibatnya, perintah kendali LED yang dikirimkan dari MQTT Explorer saat jeda tersebut tidak akan direspons sama sekali, dan koneksi MQTT berisiko terputus (*timeout*) karena fungsi client.loop() terlambat dipanggil.

2. **Cara Kerja Fungsi reconnect() Memulihkan Koneksi:**
    - **Penjelasan:** Ketika koneksi terputus (!client.connected()), fungsi reconnect() dijalankan untuk masuk ke dalam prosedur pemulihan. Fungsi ini memanggil client.connect(clientId) secara berkala. Jika berhasil terhubung kembali ke *broker*, kode di dalamnya akan langsung mengeksekusi perintah client.subscribe(topicSubscribe); untuk mendaftarkan ulang topik kendali secara otomatis, sehingga sistem dapat kembali menerima perintah dan mengirim data telemetri tanpa perlu merestart perangkat secara manual.

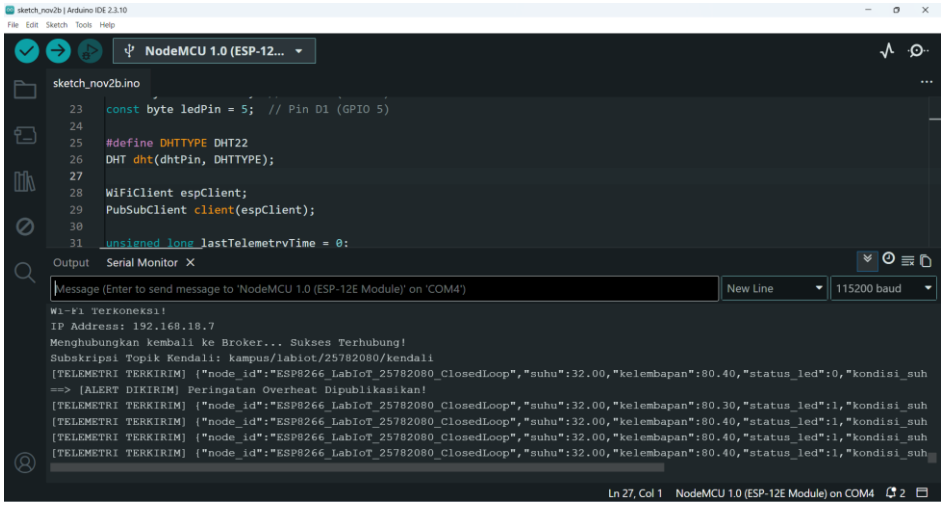
6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

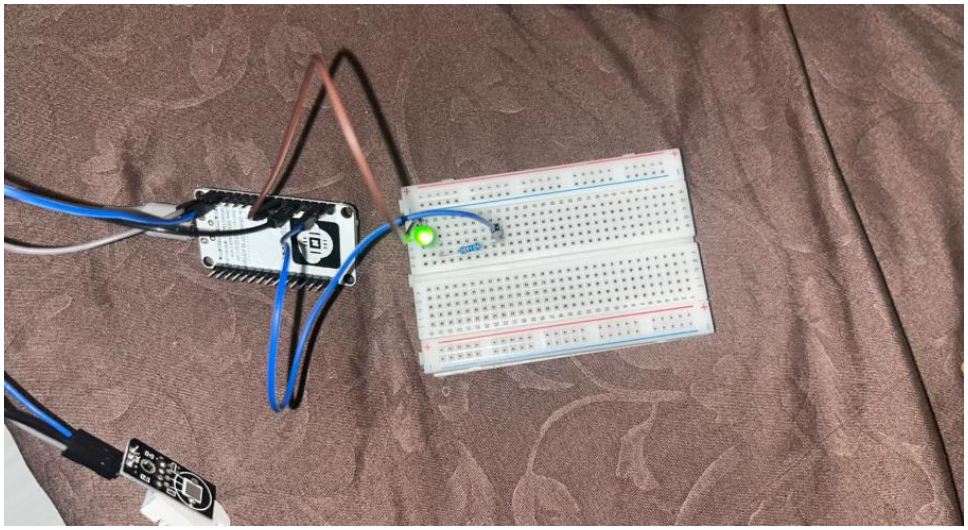
menampilkan cuplikan antarmuka perangkat lunak Arduino IDE beserta hasil keluaran (*output*) pada Serial Monitor yang merekam seluruh proses sistem *Closed-Loop Industrial Safety* secara lengkap.

Berikut adalah penjelasan detail mengenai log aktivitas yang terlihat pada Serial Monitor:

- Inisialisasi Jaringan & Koneksi MQTT:
    - Wi-Fi Terkoneksi! dan IP Address: 192.168.18.7 menunjukkan mikrokontroler NodeMCU berhasil terhubung ke jaringan Wi-Fi lokal.
    - Menghubungkan kembali ke Broker... Sukses Terhubung! menandakan sesi komunikasi dengan *broker* MQTT publik berhasil dibangun.
    - Subskripsi Topik Kendali: kampus/labiot/25782080/kendali menunjukkan perangkat siap mendengarkan perintah kendali jarak jauh dari operator luar.

- Eksekusi Edge Safety Engine & Overheat Alert:
  - [TELEMETRI TERKIRIM] ... "suhu":32.00, ... "status_led":0, "kondisi_suh...: Menampilkan pengiriman data telemetri berkala di mana suhu terbaca sebesar 32.00°C (melebihi ambang batas aman 31.0°C).
  - ==> [ALERT DIKIRIM] Peringatan Overheat Dipublikasikan!: Sistem secara otomatis mendeteksi kondisi bahaya lokal dan mempublikasikan peringatan instan ke topik *alert* (kampus/labiot/25782080/alert) tanpa menunggu perintah luar.
  - Baris telemetri berikutnya menunjukkan status_led: 1, yang membuktikan bahwa perangkat keras LED langsung menyala secara otomatis sebagai respons pengaman (*safety actuation*).

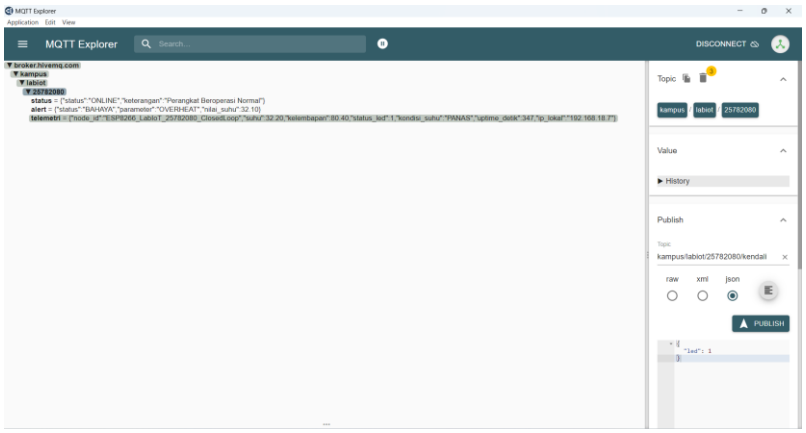




menampilkan antarmuka aplikasi **MQTT Explorer** yang terhubung dengan sempurna ke *broker* MQTT publik (broker.hivemq.com), di mana seluruh struktur hierarki topik dan data dari sistem *Closed-Loop Industrial Safety* Anda berhasil tampil secara lengkap dan sesuai dengan target praktikum.

Berikut adalah penjelasan detail mengenai isi dari tampilan tersebut:

- **Panel Kiri (Topic Tree / Hierarki Topik):**
    - Menunjukkan struktur direktori data yang dikirim oleh perangkat mikrokontroler dengan NIM **25782080** di bawah folder kampus / labiot.
    - **Topik status:** Berisi payload JSON {"status": "ONLINE", "keterangan": "Perangkat Beroperasi Normal"}, yang menandakan perangkat berhasil terhubung dan aktif.
    - **Topik alert:** Berisi payload JSON {"status": "BAHAYA", "parameter": "OVERHEAT", "nilai_suhu": 32.10}, yang membuktikan bahwa *Edge Safety Engine* aktif secara otomatis ketika suhu melewati ambang batas 31.0°C.
    - **Topik telemetri:** Berisi paket data sensor berkala dalam format JSON lengkap, yaitu:
        - node_id: Identitas klien ESP8266_LabIoT_25782080_ClosedLoop.
        - suhu: Nilai suhu terukur 32.20.
        - kelembapan: Nilai kelembapan 80.40.
        - status_led: Status fisik LED bernilai 1 (menyala otomatis karena kondisi panas).
        - kondisi_suhu: Klasifikasi lingkungan bernilai "PANAS".
        - uptime_detik: Lama waktu aktif sistem 347 detik.
        - ip_lokal: Alamat IP lokal mikrokontroler 192.168.18.7.
- **Panel Kanan Bawah (Area Publish):**
    - Menunjukkan kotak konfigurasi pengiriman pesan ke topik **kampus/labiot/25782080/kendali** dengan format JSON {"led": 1} untuk pengujian kendali manual *override*.


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782080
status = {"status":"ONLINE","keterangan":"Perangkat Beroperasi Normal"}
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":32.10}
telemetri = {"node_id":"ESP8266_LabIoT_25782080_ClosedLoop","suhu":32.20,"kelembapan":80.40,"status_led":1,"kondisi_suhu":"PANAS","uptime_detik":347,"ip_lokal":"192.168.18.7"}
Topic
kampus
labiot
25782080
Value
History
Publish
Topic
kampus/labiot/25782080/kendali
raw
xml
json
PUBLISH
{
"led": 1
}